ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Abrao Soares Pereira

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A05_Sp26.Rmd"). Then change "Student Name" on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: "readxl", "ggplot2", "forecast","tseries", and "Kendall". Install these packages, if you haven't done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
# Margins setup
knitr::opts_chunk$set(echo = TRUE, tidy.opts = list(width.cutoff = 50), tidy = TRUE)


#Load/install required package here
library(forecast)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(ggplot2)
library(Kendall)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
library(tidyverse)  #load this package so yon clean the data frame using pipes
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.5.1
## v forcats 1.0.0     v tibble  3.3.0
## v purrr   1.1.0     v tidyr   1.3.2
## v readr   2.1.5
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(readxl)
```

Consider the same data you used for A04 from the spreadsheet "Table_10.1_Renewable_Energy_Production_and_Consump
The data comes from the US Energy Information and Administration and corresponds to the December 2025 Monthly Energy Review.

```r
#Importing data set - using readxl package
energy_data <- read_excel(
  path= "/Users/abraopereira/Documents/ENVIRON 797 - TSA Spring 2026/TSA_Spring 2026/Data/Table_10.1_Re
  skip = 12,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
read_col_names <- read_excel(
  path="/Users/abraopereira/Documents/ENVIRON 797 - TSA Spring 2026/TSA_Spring 2026/Data/Table_10.1_Ren
  skip = 10,
  n_max = 1,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)

nobs=nrow(energy_data)
nvar=ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month               `Wood Energy Production` `Biofuels Production`
##   <dttm>                                  <dbl> <chr>
## 1 1973-01-01 00:00:00                      130. Not Available
## 2 1973-02-01 00:00:00                      117. Not Available
## 3 1973-03-01 00:00:00                      130. Not Available
## 4 1973-04-01 00:00:00                      125. Not Available
## 5 1973-05-01 00:00:00                      130. Not Available
## 6 1973-06-01 00:00:00                      125. Not Available
## # i 11 more variables: `Total Biomass Energy Production` <dbl>,
## #   `Total Renewable Energy Production` <dbl>,
## #   `Hydroelectric Power Consumption` <dbl>,
## #   `Geothermal Energy Consumption` <dbl>, `Solar Energy Consumption` <chr>,
## #   `Wind Energy Consumption` <chr>, `Wood Energy Consumption` <dbl>,
## #   `Waste Energy Consumption` <dbl>, `Biofuels Consumption` <chr>,
## #   `Total Biomass Energy Consumption` <dbl>, ...
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date, Solar Energy Consumption and Wind Energy Consumption**. Check the class of columns, you will see that they are stored are characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as "Not Available". Convert the data to numeric, the "Not Available" will became NAs.

You may either filter out the "Not Available" rows and then convert the column to numeric or convert first and then remove missing values using drop_na() (or na.omit()). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is because they only happen in the beginning of the series!

```r
# Roll in data
enery_data_clean <- energy_data %>%
  select(Month,
         `Solar Energy Consumption`,
         `Wind Energy Consumption`) %>%

# Convert character into numeric and clean NAs
mutate(
  Month = as.Date(Month),
  `Solar Energy Consumption`= as.numeric(`Solar Energy Consumption`),
  `Wind Energy Consumption`= as.numeric(`Wind Energy Consumption`)
) %>%
  drop_na()
```

```
## Warning: There were 2 warnings in `mutate()`.
## The first warning was:
## i In argument: `Solar Energy Consumption = as.numeric(`Solar Energy
##   Consumption`)`.
## Caused by warning:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

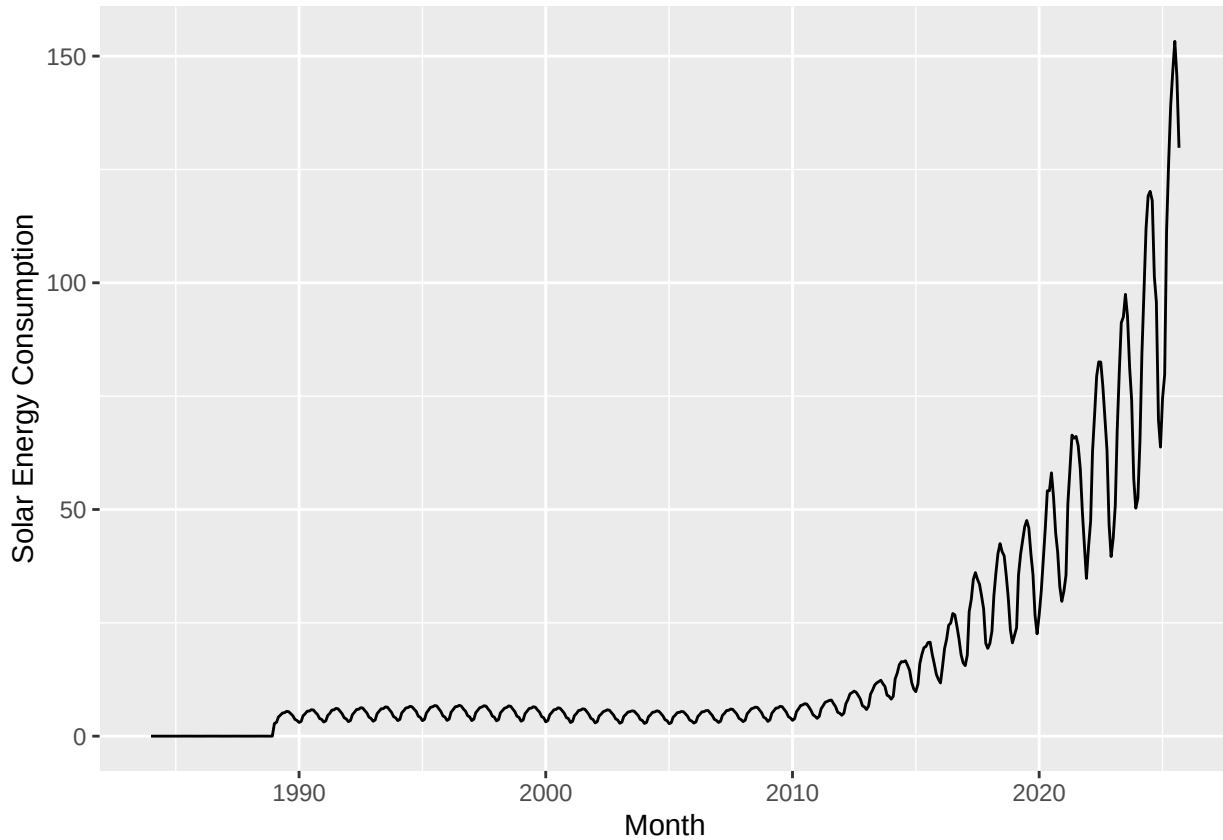
```r
head(enery_data_clean)
```

```
## # A tibble: 6 x 3
##   Month      `Solar Energy Consumption` `Wind Energy Consumption`
##   <date>                          <dbl>                     <dbl>
## 1 1984-01-01                      0                         0
## 2 1984-02-01                      0                         0.001
## 3 1984-03-01                      0.001                     0.001
## 4 1984-04-01                      0.001                     0.002
## 5 1984-05-01                      0.002                     0.003
## 6 1984-06-01                      0.003                     0.002
```

Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")")`

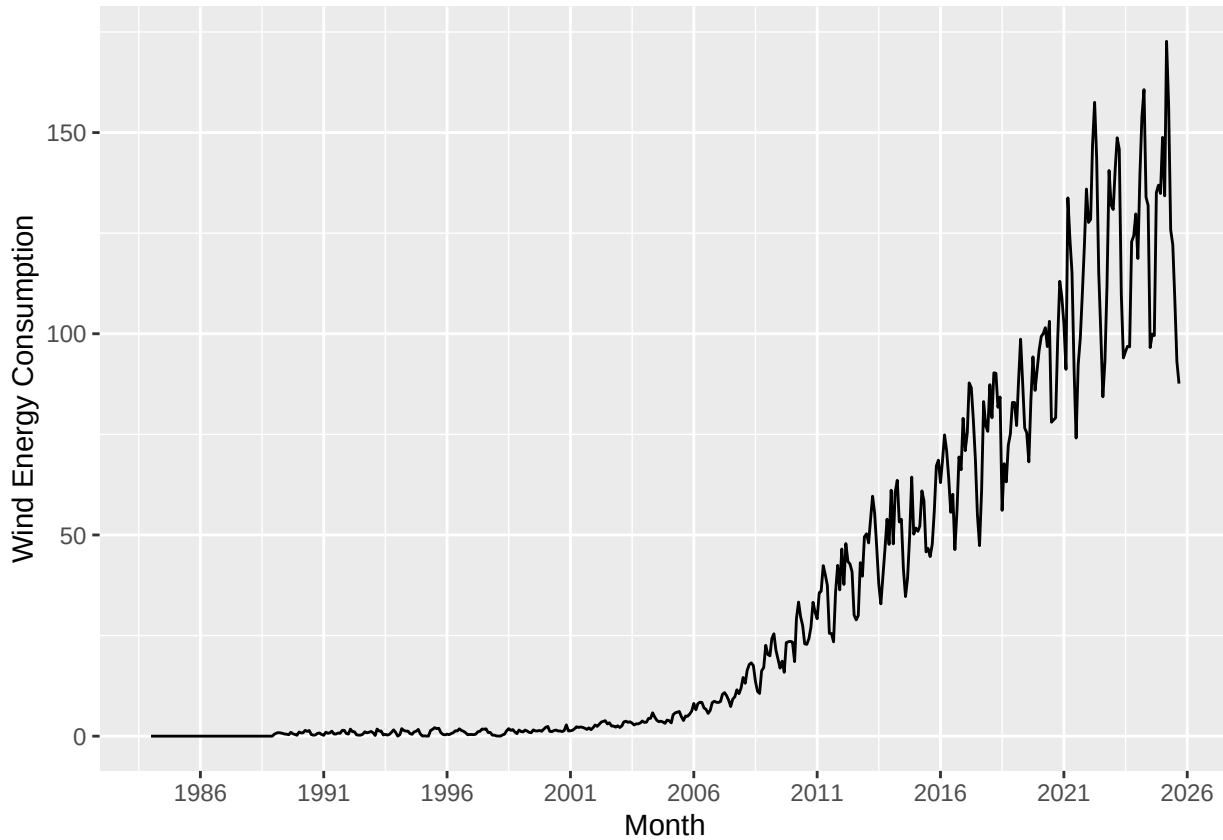
```r
# Plot for wind energy
ggplot(enery_data_clean, aes( x = Month, y = `Solar Energy Consumption`)) +
  geom_line()
```




```
ylab("Solar Energy Consumption") +
xlab("Year") +
scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

```
## NULL
```

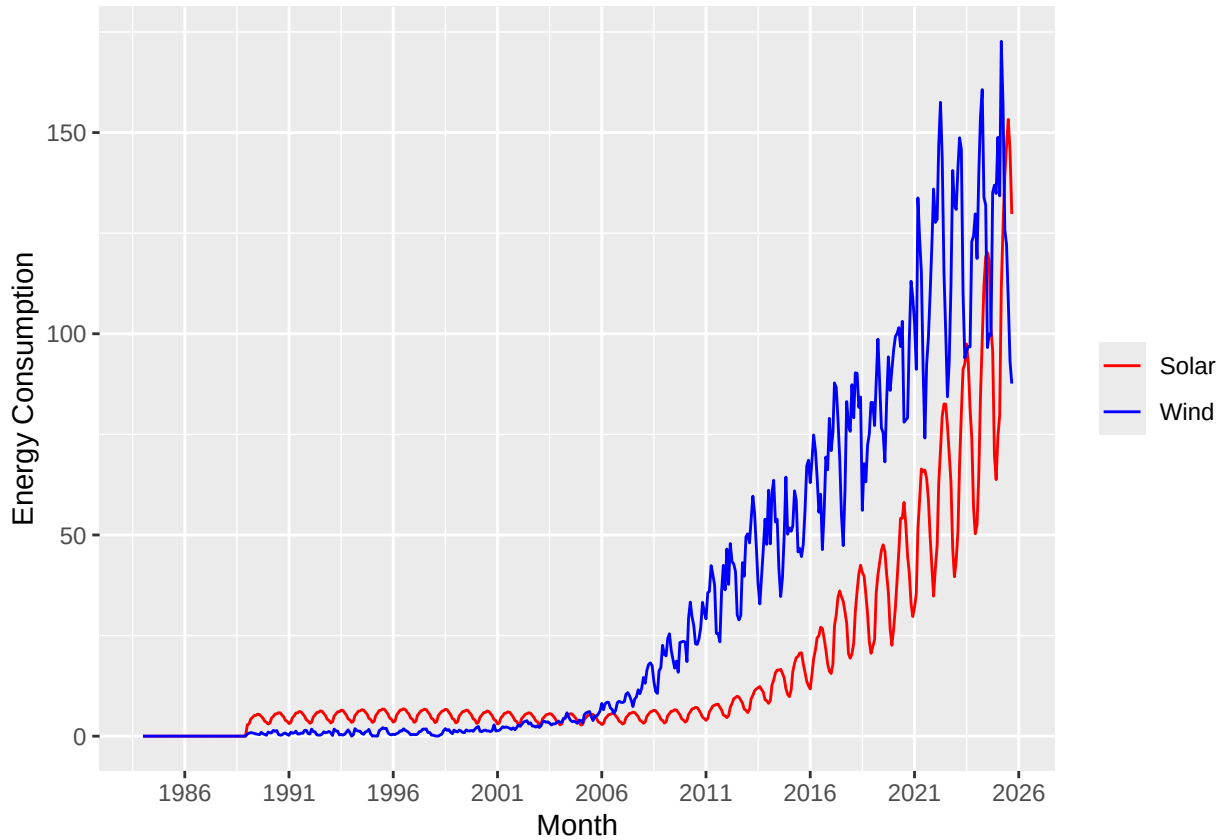
```
# Plot for energy
ggplot(enery_data_clean, aes(x = Month, y = `Wind Energy Consumption`)) +
  geom_line() +
  ylab("Wind Energy Consumption") + scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```



Q3

Now plot both series in the same graph, also using ggplot(). Use function `scale_color_manual()` to manually add a legend to ggplot. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption)`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```
ggplot(enery_data_clean, aes(x = Month)) +
  geom_line(aes(y = `Solar Energy Consumption`, colour = "Solar")) +
  geom_line(aes(y = `Wind Energy Consumption`,  colour = "Wind")) +
  scale_color_manual(values = c("Solar" = "red", "Wind" = "blue")) +
  ylab("Energy Consumption") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(color = NULL)
```



Decomposing the time series

The stats package has a function called decompose(). This function only take time series object. As the name says the decompose function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

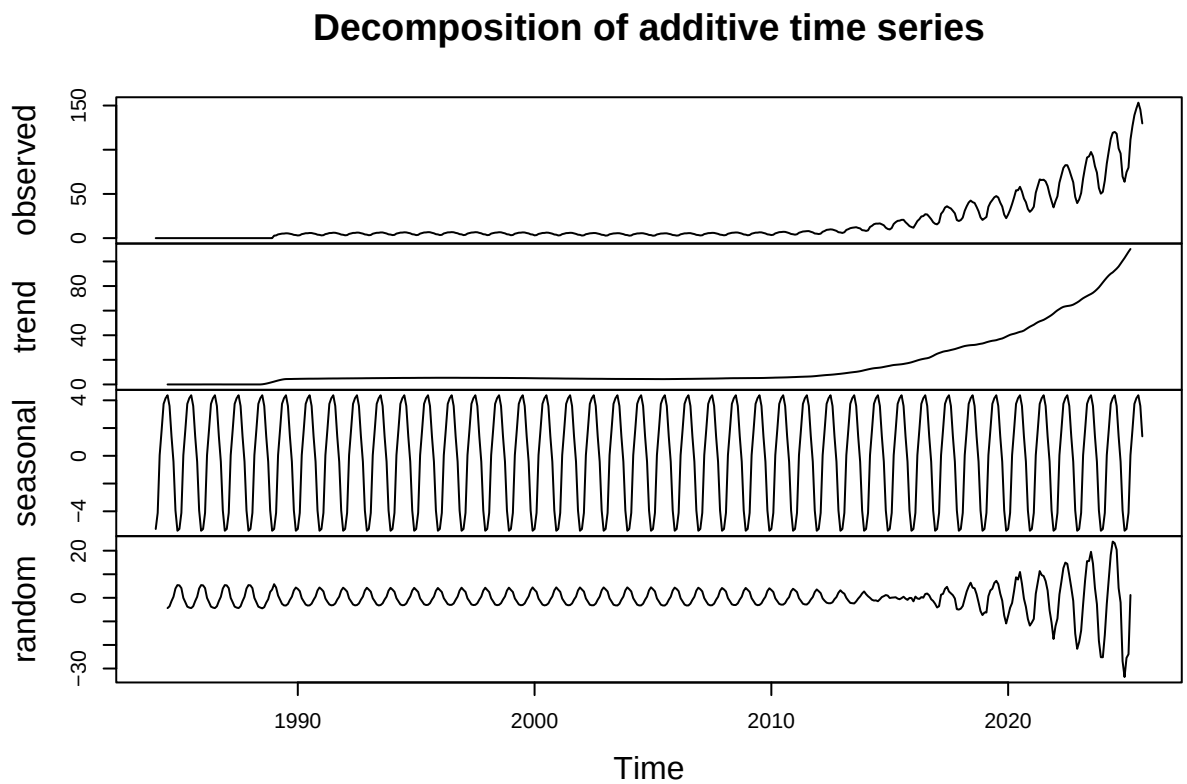
Additional info on `decompose()`.

1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
2) The trend is not a straight line because it uses a moving average method to detect trend.
3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

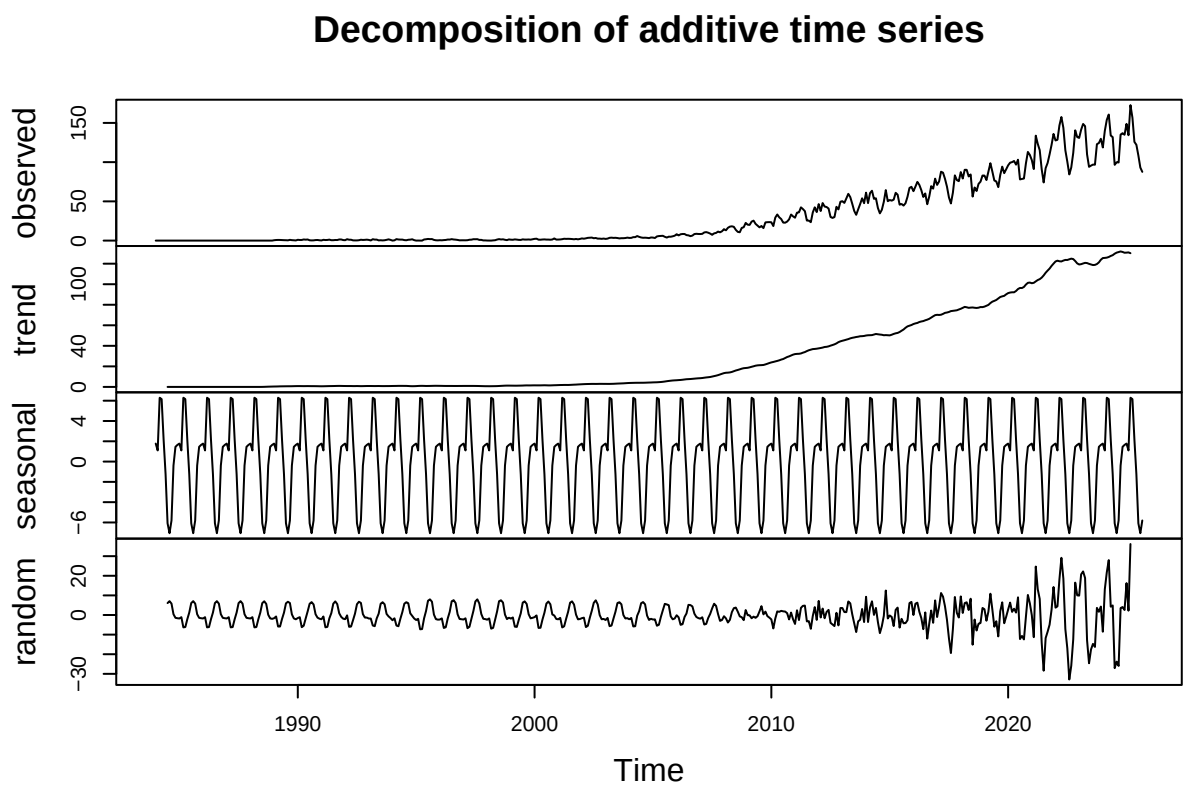
Q4

Transform wind and solar series into a time series object and apply the decompose function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```r
# Use automate process
start_year  <- year(min(enery_data_clean$Month))
start_month <- month(min(enery_data_clean$Month))

# Time series for the solar and wind energy
solar_timser <- ts( enery_data_clean$`Solar Energy Consumption`, start = c(start_year, start_month),fre

wind_timser <- ts(enery_data_clean$`Wind Energy Consumption`,start = c(start_year, start_month),frequen

# Apply decompose function
solar_dec_add <- decompose(solar_timser, type = "additive")
wind_dec_add  <- decompose(wind_timser,  type = "additive")

plot(solar_dec_add)
```



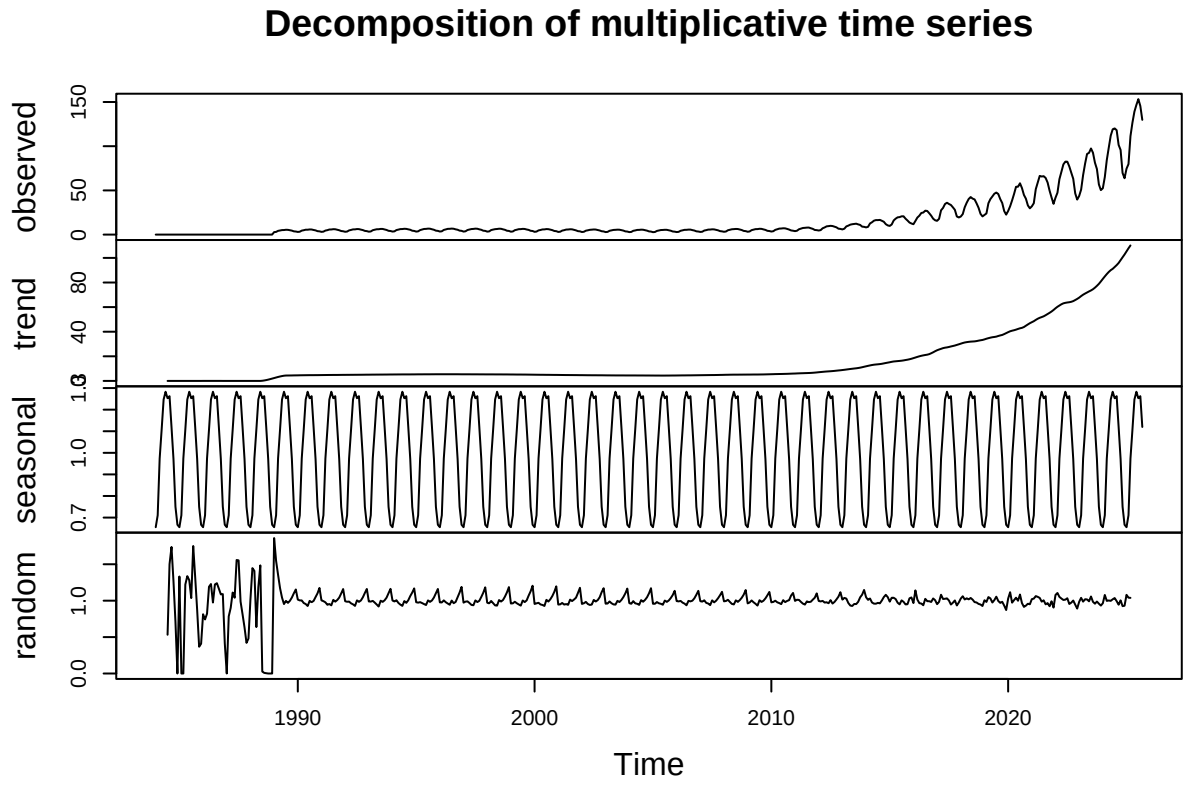

```r
plot(wind_dec_add)
```



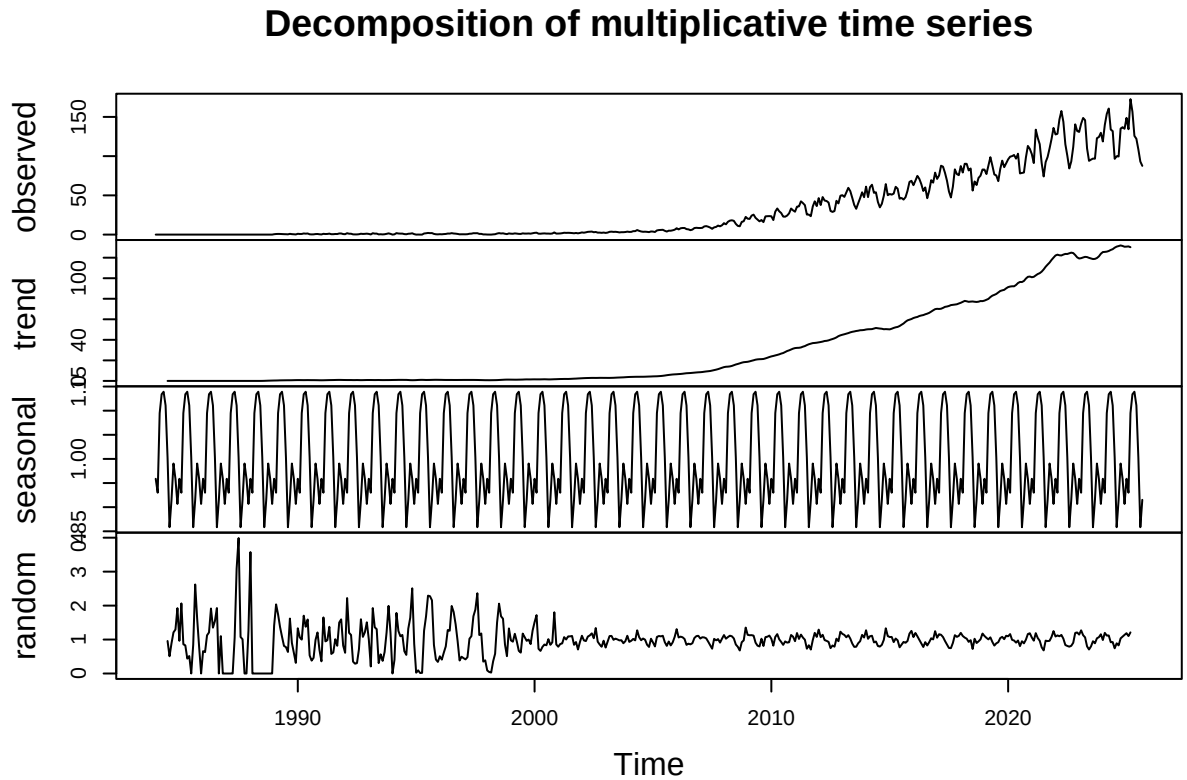
Q5

Use the decompose function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

```
solar_multiplicative <- decompose(solar_timser, type = "multiplicative")
wind_multiplicative  <- decompose(wind_timser,  type = "multiplicative")

# Plot the new results
plot(solar_multiplicative)
```




```r
plot(wind_multiplicative)
```



Q6

When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

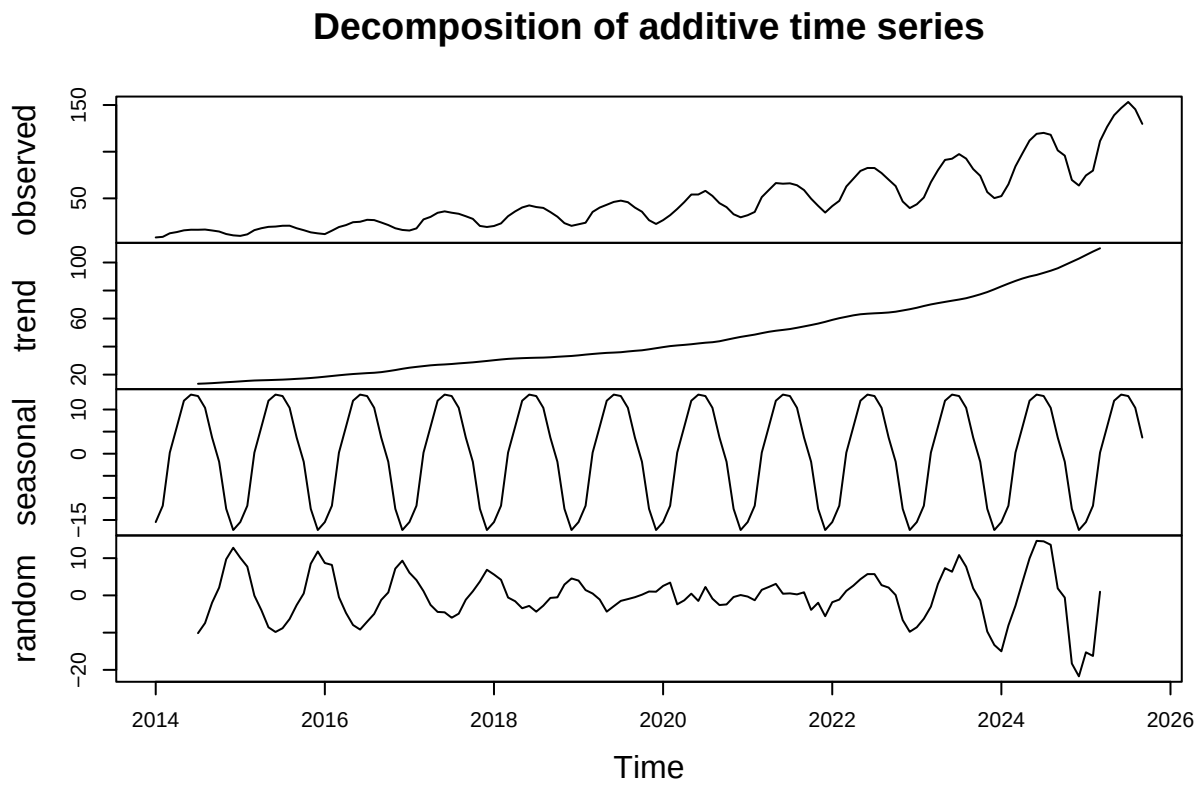
> Answer: Theoretically, historical data is not fitting in the model because it technically contains no relevant information, mostly because there are no information to use to train the model, hence we do not need it. However, the recent data in earl or late 90s through 20s also the same but the data within a couple years of it represent more current data.

Q7

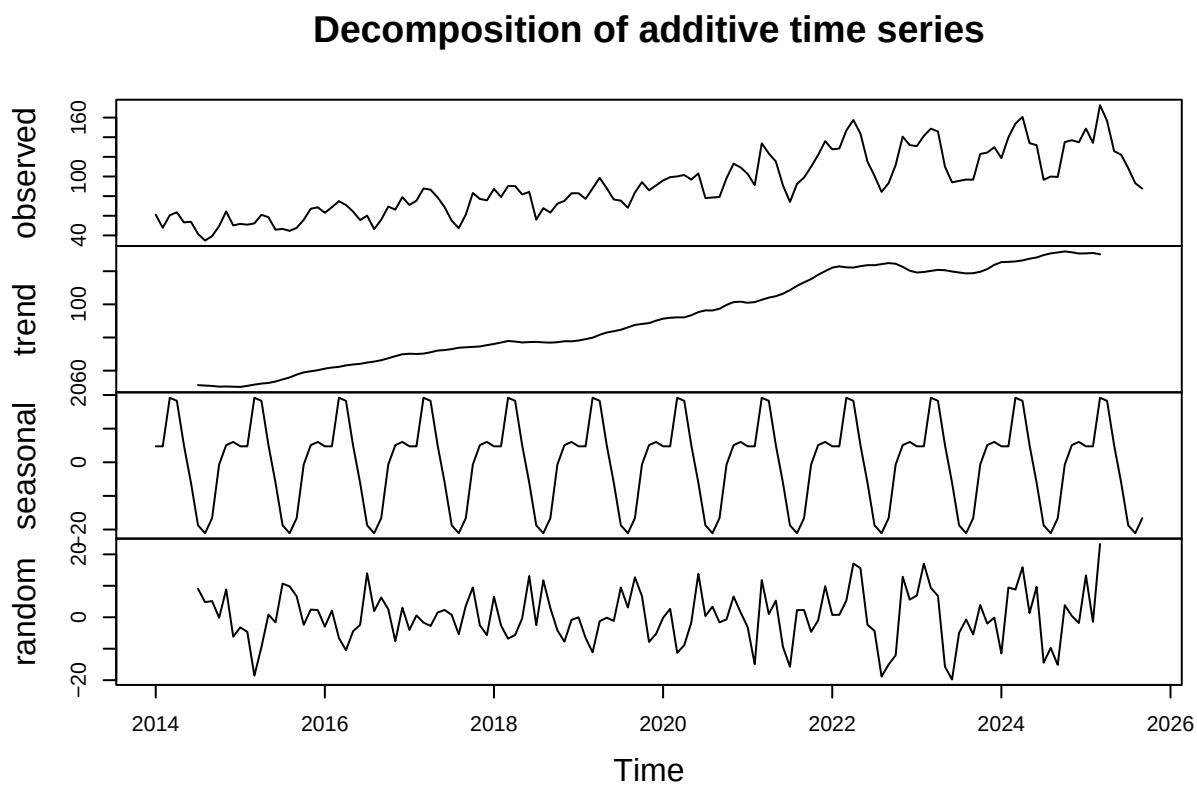
Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, .i.e, `filter(xxxx, year(Date) >= 2014 )`. Apply the decompose function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```
recent_energy <- enery_data_clean %>%
  filter(year(Month) >= 2014)

recent_solar_timser <- ts(recent_energy$`Solar Energy Consumption`, start = c(2014, 1), frequency = 12)

recent_wind_timser <- ts(recent_energy$`Wind Energy Consumption`, start = c(2014, 1), frequency = 12)
```

```r
recent_decomp_solar <- decompose(recent_solar_timser, type = "additive")
recent_decomp_wind  <- decompose(recent_wind_timser,  type = "additive")

plot(recent_decomp_solar)
```




```r
plot(recent_decomp_wind)
```



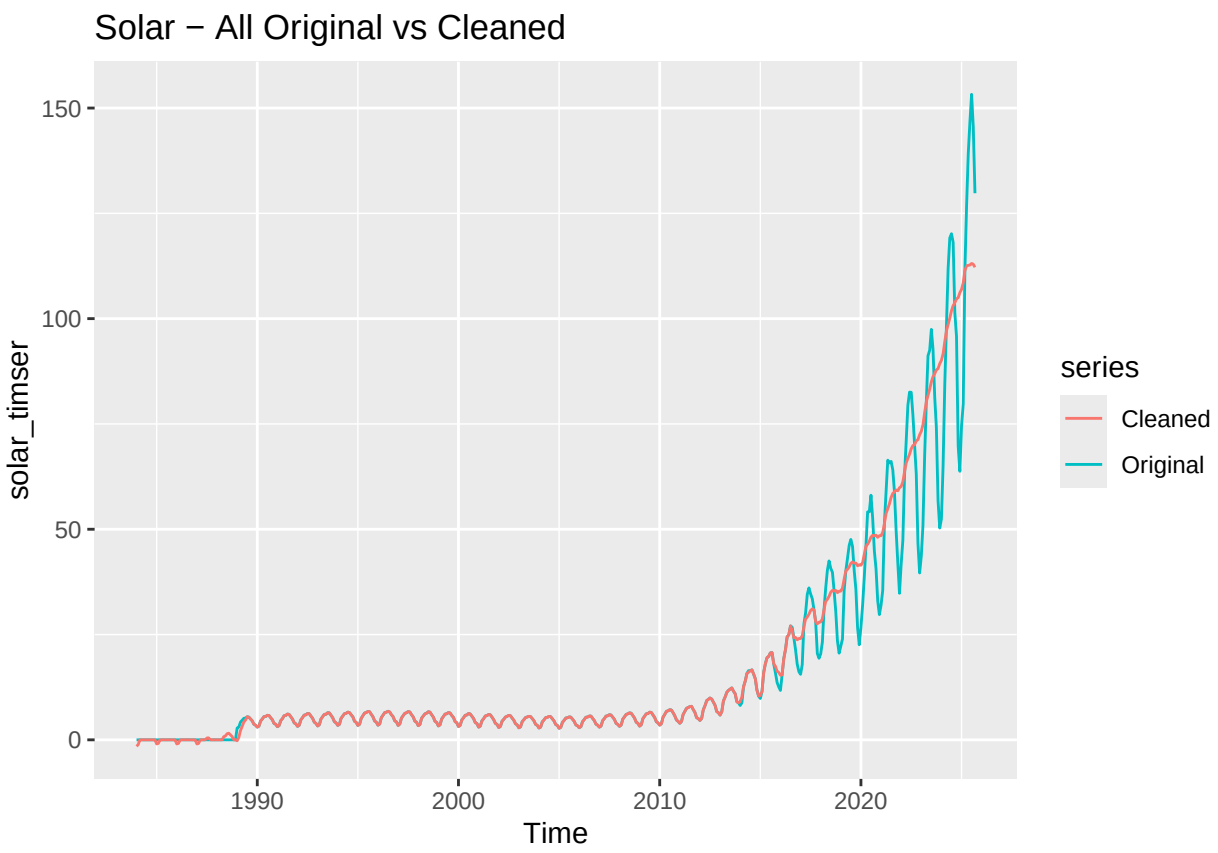
Answer: For the 2014 onward, the trend still shows the recent growth period clearly and smoothly. The seasonal pattern stay consistent across years.

Identify and Remove outliers

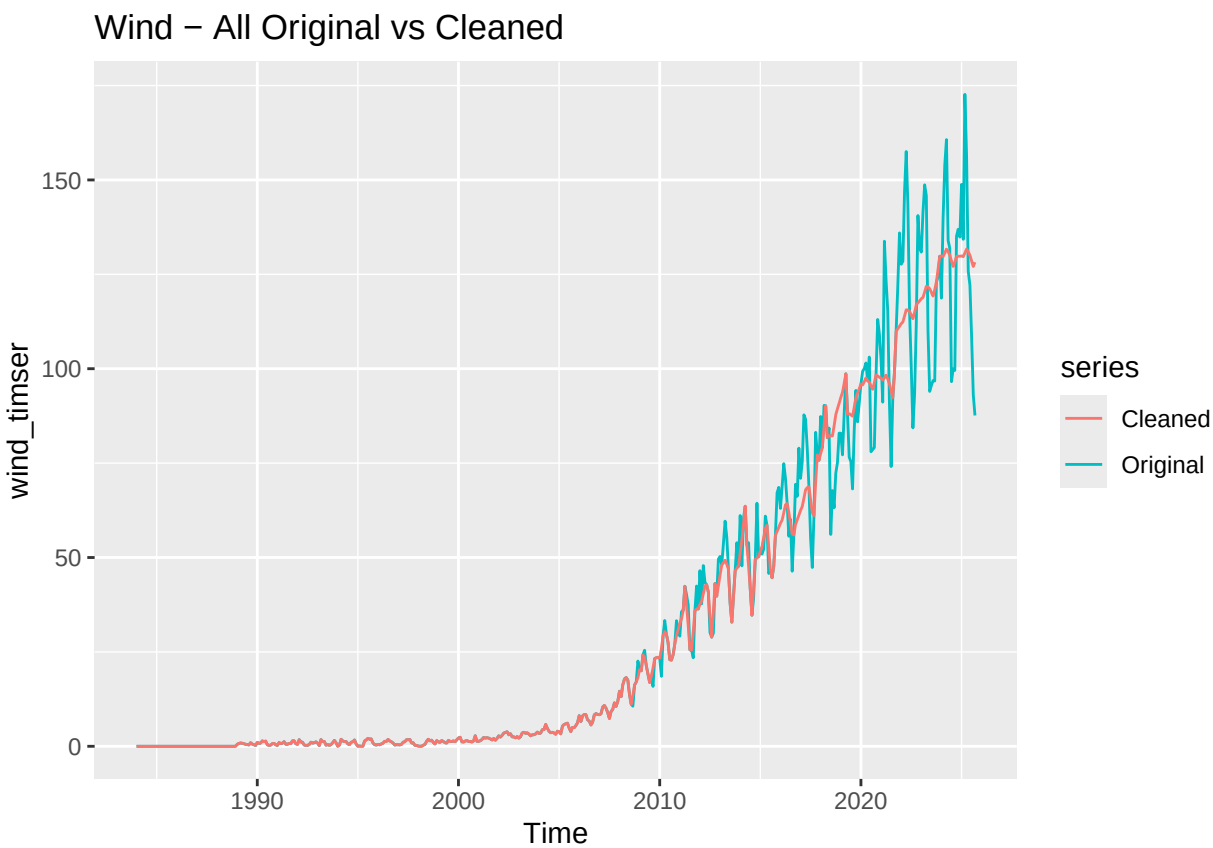
Q8

Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

```
solar_original <- tsclean(solar_timser)
wind_original  <- tsclean(wind_timser)

autoplot(solar_timser, series = "Original") +
  autolayer(solar_original, series = "Cleaned") +
  ggtitle("Solar - All Original vs Cleaned")
```




```
autoplot(wind_timser, series = "Original") +
  autolayer(wind_original, series = "Cleaned") +
  ggtitle("Wind - All Original vs Cleaned")
```

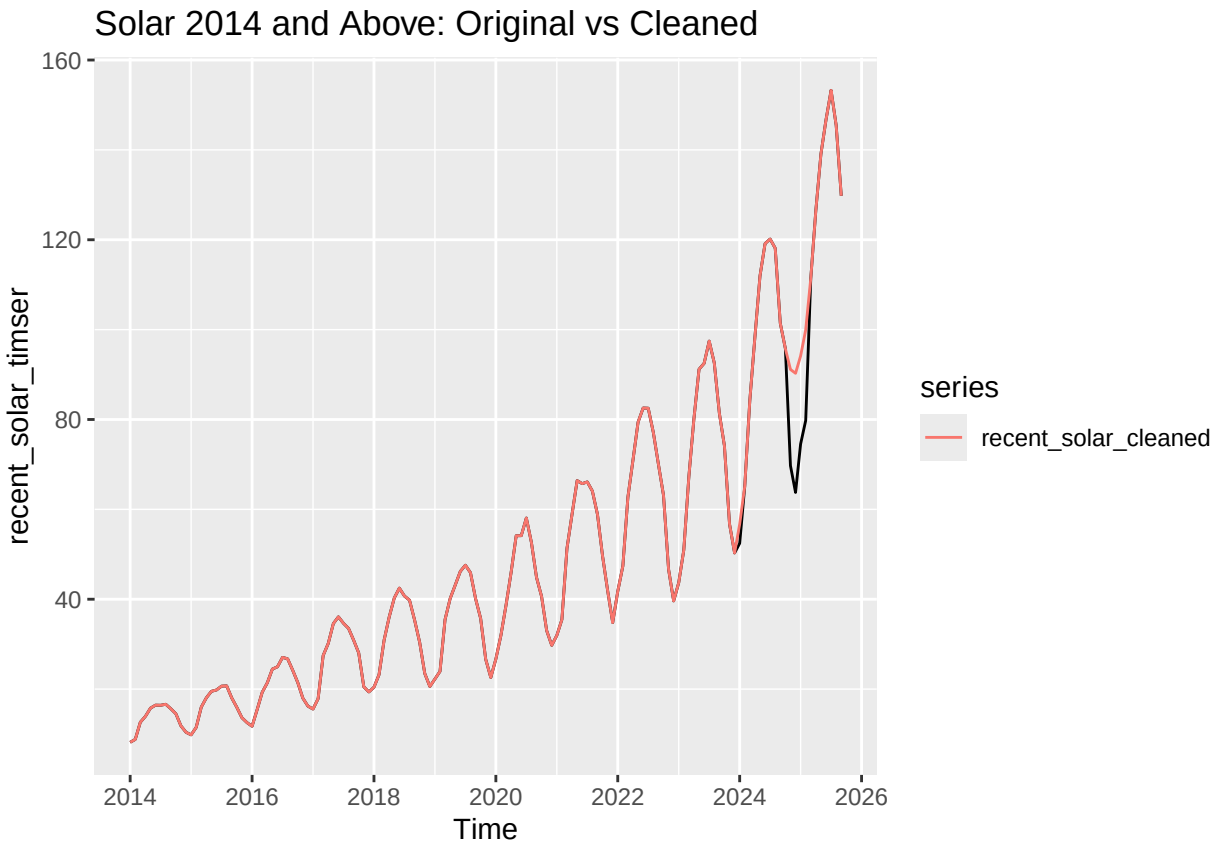


Answer: The function did remove outliers from both the Solar and Wind series. In the in the earliest data from the 80s, then the groeth and increasing of the last five years are flagged as anomalies compared to the historical baseline. This is most visible in the Wind plot, where the orange "Cleaned" line significantly smooths out the teal "Original" spikes.

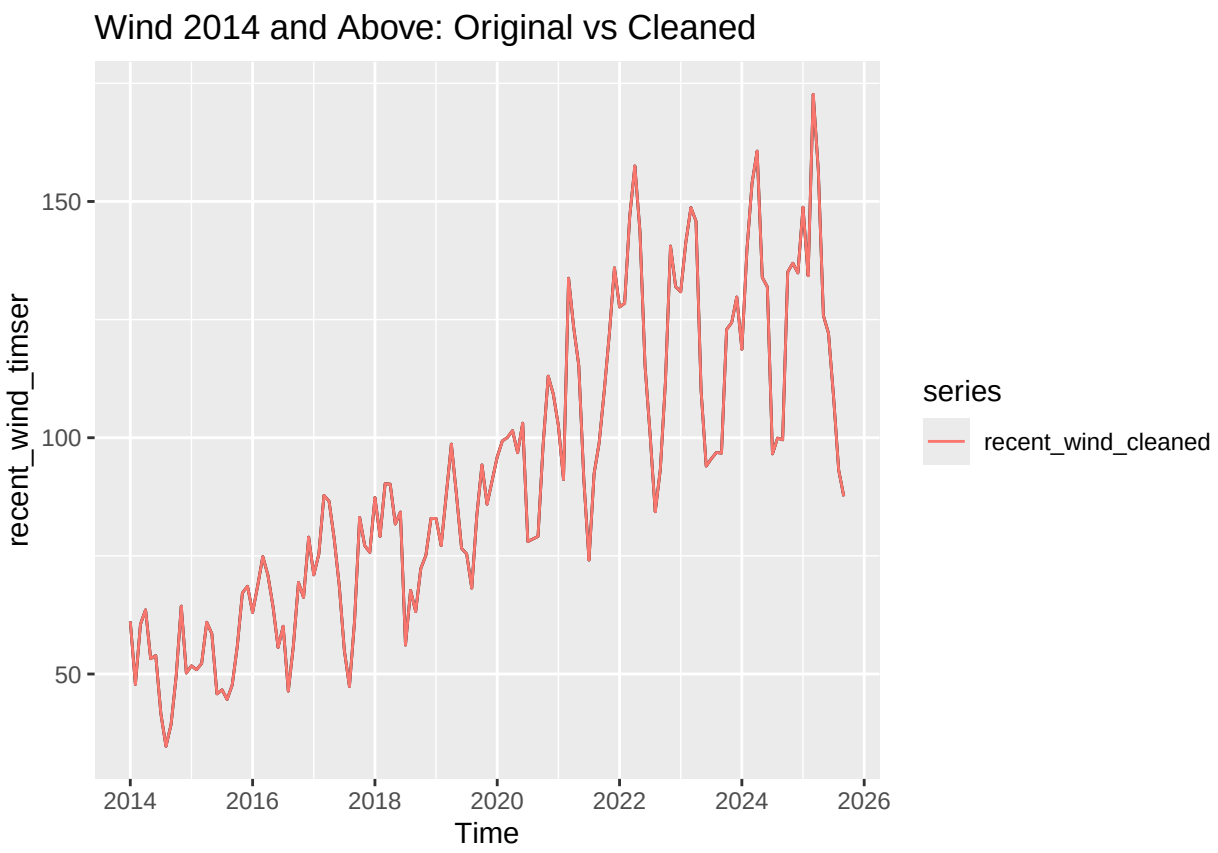
Q9

Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what `autoplot()` again what happened now? Did the function removed any outliers from the series?

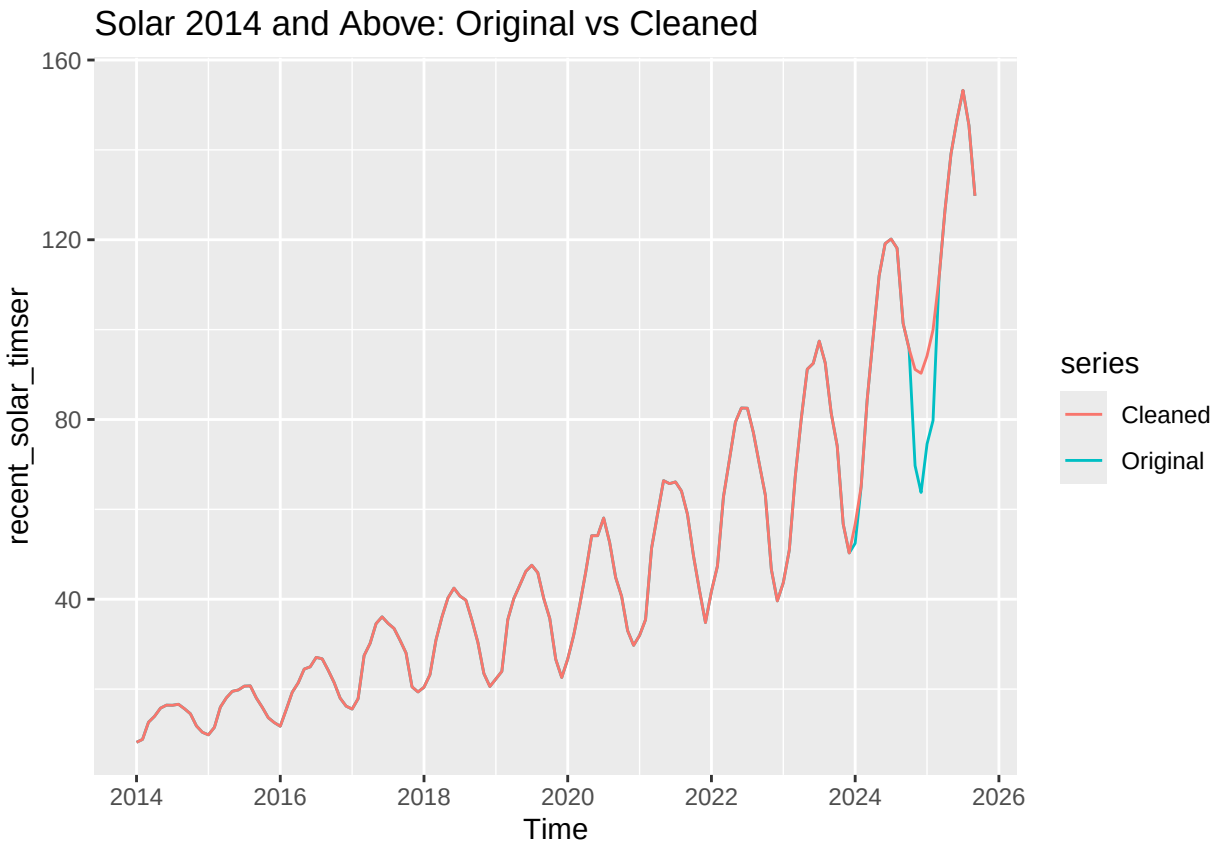
```
recent_solar_cleaned <- tsclean(recent_solar_timser)
recent_wind_cleaned  <- tsclean(recent_wind_timser)

# Auto plots
autoplot(recent_solar_timser) +
  autolayer(recent_solar_cleaned) +
  ggtitle("Solar 2014 and Above: Original vs Cleaned")
```




```
autoplot(recent_wind_timser) +
  autolayer(recent_wind_cleaned) +
  ggtitle("Wind 2014 and Above: Original vs Cleaned")
```




```
autoplot(recent_solar_timser, series = "Original") +
  autolayer(recent_solar_cleaned, series = "Cleaned") +
  ggtitle("Solar 2014 and Above: Original vs Cleaned")
```



Answer: The function was much more smooth and clear path. For Wind, it found no outliers, resulting in perfectly overlapping lines. For Solar, it identified and corrected only one minor outlier roughly near the year 2025. This shows that modern data is more consistent,mostly maybe due to more accurate data collection and organization.